Quick Sort Algorithm

Explaination?
Intuition
↓
Pseudocode
↓
Dry run
↓
Compiler
↓
Time complexity

- use in sorting
- will sort in ascending order
- can also used to sort in descending order. with some tweak.

Quick Sort > Merge Sort

[4 6 2 5 7 9 1 3]

Step 1: Pick a pivot & place it in its correct place in the sorted array

Pivot can be;

a) 1st element in the array

b) last element in the array

c) Median of the array

d) random element of the array.

unsorted array

4 6 2 5 7 9 1 3

1 2 3 4 5 6 7 9

correct Place

Sorted array

Pivot can be any thing.

Step 2: Smaller on the left larger on the right

smaller greater

1 step

4 6 2 5 7 9 1 3

2 1 3 4 6 5 7 9

greater → unsorted

smaller → unsorted

2 1 3 6 5 7 9

1 2 3

5 6 7 9

Correct order

7 9

7 9

1 2 3 4 5 6 7 9

Pseudo Code

index

4	6	2	5	7	9	1	3
0	1	2	3	4	5	6	7

low index ↑ High index ↑

pivot = a[low]

i > pivot j < pivot

4	6	2	5	7	9	1	3
0	1	2	3	4	5	6	7

~~i=0~~ i=1 j=7

Swap this

i = 3 j = 6

~~i~~ ~~i~~ i > pivot j < pivot ~~j~~

~~4~~	3	2	5	7	9	1	6
0	1	2	3	4	5	6	7

Swap

i = 4

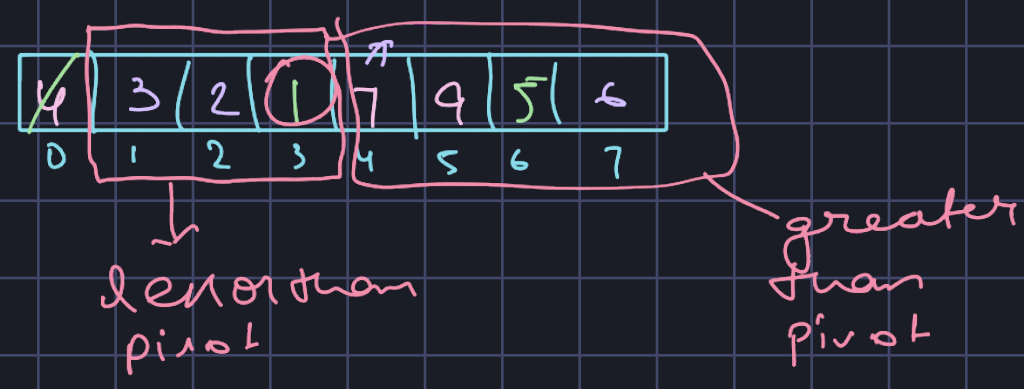
~~i~~ i > pivot ~~j~~ ~~j~~

~~4~~	~~6~~	~~2~~	1	7	9	5	~~3~~
0	1	2	3	4	5	6	7

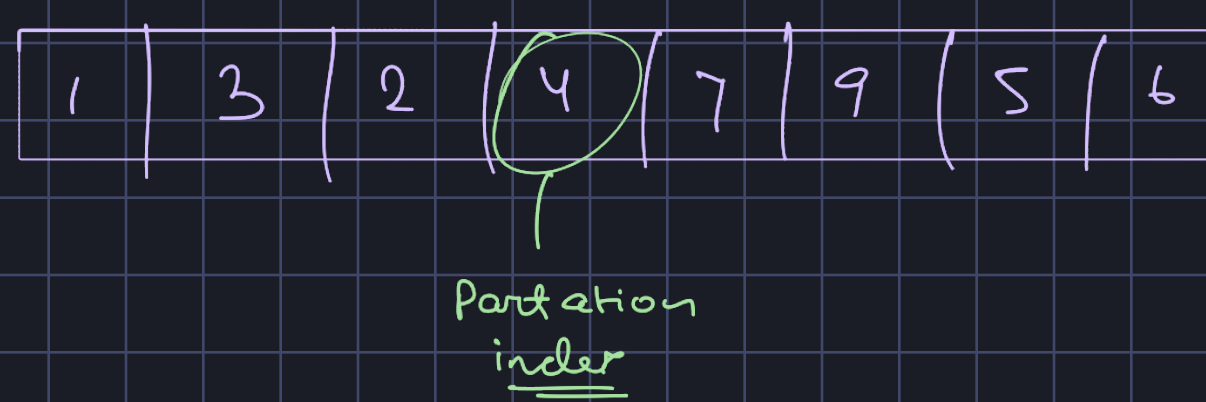
j < pivot

j = 3

when i and j crossed
you do not swap the number



Since you stop at 1 then place the pivot where just i and j crossed.



Pseudo code;

```
quickSort(arr, low, high){
    if(low < high){
        partationIndex = f(arr, low, high);
        quickSort(arr, low, pIndex - 1)
        quickSort(arr, pIndex + 1, high)
    }
}
```

```
int function(arr, low, high){

    pivot = arr[low];
    i = low;
    j = high;

    while (i<j){
        while (arr[i]<=arr[pivot] &&
                    i <= high-1 ){
            i++;
        }
        while (arr[j] > arr[pivot] &&
                    j >= low+1){
            j--;
        }

        if (i<j){
            swap(arr[i], arr[j]) }
        swap(arr[low], arr[j]);
        return j;
    }
}
```